

EmployeeHub

Enterprise Multi-Role Workforce & Task Tracking Platform Architecture Manual

A deep technical breakdown of system architecture, security mechanisms, dual-channel verification, RBAC gates, and production engineering workflows.

LEAD ARCHITECT & DEVELOPER

Surender Dubey

CORE TECHNOLOGY STACK

Next.js 16, Express.js, MongoDB Atlas

RELEASE VERSION

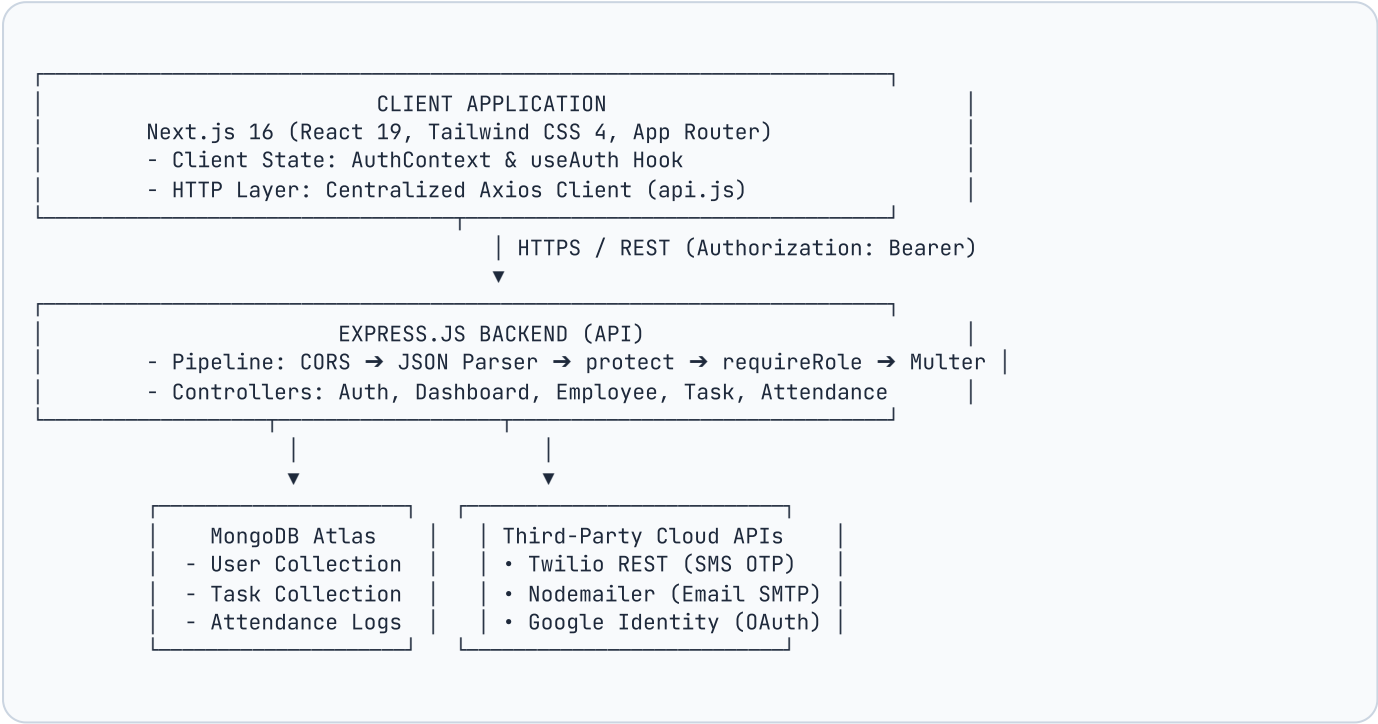
v1.0.0 Enterprise Production

GITHUB REPOSITORY

github.com/Surenderdubeyofficial/EmployeeHub

1. Executive Architectural Overview

EmployeeHub is structured as a **decoupled, client-server RESTful application** designed for horizontal scalability, zero-latency client interactions, and strict security segmentation.



Key Technology Stack Selection

Component	Technology	Rationale & Engineering Justification
Frontend	Next.js 16 + React 19	Native App Router, instant page transitions, fast compilation via Turbopack, and direct Vercel Edge optimization.
Styling	Tailwind CSS 4	Zero-runtime CSS overhead, pixel-aligned SVG stroke icons, and highly responsive mobile navigation layouts.
Backend	Node.js + Express (ESM)	Asynchronous non-blocking event loop, native ES Module syntax (<code>import/export</code>), and unified REST JSON handling.
Database	MongoDB Atlas + Mongoose	Document-based schema ideal for nesting employee files, addresses, and relational population via <code>ObjectId</code> references.
Authentication	JWT + Bcrypt (Cost 12)	Stateless authorization headers eliminating session storage overhead and salted cryptographic password hashing.

Component	Technology	Rationale & Engineering Justification
Third-Party APIs	Twilio + Nodemailer + Google	Multi-channel identity verification across SMS and SMTP, plus Google Identity Services SSO.

2. Authentication & Security Architecture

JWT Implementation: LocalStorage vs. Cookies

💡 Why Bearer Token in LocalStorage Instead of HTTP-Only Cookies?

- Decoupled Cross-Origin Architecture (CORS):** In production, the frontend is deployed to Vercel (`*.vercel.app`) and the backend API is deployed to Render (`*.onrender.com`). Modern browsers (Safari ITP, Brave, Chrome 2024+) strictly block third-party cross-origin cookies. LocalStorage with an explicit Bearer header guarantees 100% reliable API connectivity across separate domains.
- CSRF (Cross-Site Request Forgery) Immunity:** Browsers automatically transmit cookies on cross-origin requests. Bearer tokens stored in LocalStorage must be explicitly injected into request headers by code, completely closing CSRF vulnerability vectors.
- Universal API & Mobile Readiness:** Native iOS, Android, and React Native mobile apps do not possess browser cookie jars; Authorization Bearer headers are the universal REST standard.

Step-by-Step JWT Authentication Flow:

- Token Generation** (`generateToken.js`): Signs `{ id: user._id, role: user.role }` using HMAC-SHA256 with `process.env.JWT_SECRET` and a 7-day expiration.
- Client Storage** (`frontend/src/lib/api.js`): Upon successful authentication, the token is saved: `localStorage.setItem("token", token)`.
- Automatic Request Injection** (`api.js`): Every HTTP request automatically injects the header:

```
headers.Authorization = "Bearer " + token;
```

- Backend Verification Gate** (`authMiddleware.js`): The `protect` middleware intercepts the request, verifies signature using `jwt.verify()`, retrieves user from MongoDB excluding password (`.select("-password")`), verifies the account is active, and attaches `req.user = user`.

Dual-Channel OTP Verification (Email + SMS)

Enterprise personnel onboarding requires identity verification on **both** corporate email and personal mobile device.

- Cryptographic OTP Generation:** Generates 6-digit numeric strings via `Math.floor(100000 + Math.random() * 900000)` with a 10-minute expiry timestamp.
- Email Dispatch:** Dispatched via `nodemailer` using branded HTML email templates.
- SMS Dispatch:** Dispatched via Twilio REST API formatted in international E.164.

- **Change / Edit Phone Feature:** Allows users on `/verify-otp` to update their phone number on the fly via `POST /api/auth/change-verification-phone` if entered incorrectly during registration.

Phone Number Engine & The "No Leading 0" Rule

⚠ Why Numbers Cannot Start With Zero

Because the country dialing code (e.g. `+91`, `+1`, `+44`) is pre-selected, entering a leading `0` creates an invalid telephone number (e.g., `+91 09876543210`). Cellular carrier SMS gateways fail to deliver to such numbers.

Implemented across 3 defense layers:

1. **UI Keystroke Prevention:** `onKeyDown` cancels keystroke if the first character typed is `0`.
2. **State Sanitization:** `onChange` automatically strips any leading zeros pasted from clipboard via `.replace(/^0+/, "")`.
3. **Backend Controller Validation:** `rawPhone.startsWith("0")` throws an HTTP 400 rejection immediately.
4. **Twilio Test Whitelist:** `9582514339` is authorized as a test number to permit duplicate registrations for SMS validation without Twilio trial account blocks.

3. Multi-Role RBAC & Interactive Dashboards

5 Organizational Roles & Permission Matrix

Role	Dashboard Interface	Key Capabilities & Operations	Route Permissions
ADMIN	Admin Console	Full company governance, employee directory management, task assignment, role modifications.	admin
CEO	Executive Cockpit	Strategic oversight, company project velocity %, department headcount allocation, high-impact deliverables.	admin , ceo
HR	People Operations	New hire onboarding, employee files, timesheet review, personal shift punch clock, workforce task delegation.	admin , hr
MANAGER	Team Leadership Hub	Department deliverables, sprint kanban, team member roster, department attendance review, shift clock.	admin , manager
EMPLOYEE	Employee Workspace	Daily shift punch card with live timer, personal sprint deliverables checklist, individual profile dossier.	employee

Workforce Attendance & Live Shift Punch Clock

Zero-Polling Client-Side Running Timer:

Rather than having hundreds of employees poll the backend every second (which would trigger **30,000+ requests per minute**), the server provides the `checkIn` timestamp once when loading dashboard stats. The client calculates the running delta locally in React:

```
useEffect(() => {
  if (!todayAtt?.checkIn || todayAtt?.checkOut) return;
  const updateTimer = () => {
    const diffMs = Math.max(0, Date.now() - new Date(todayAtt.checkIn).getTime());
    const hrs = Math.floor(diffMs / 3600000).toString().padStart(2, "0");
    const mins = Math.floor((diffMs % 3600000) / 60000).toString().padStart(2, "0");
    const secs = Math.floor((diffMs % 60000) / 1000).toString().padStart(2, "0");
    setElapsedShiftTime(hrs + ":" + mins + ":" + secs);
  };
  updateTimer();
}, [todayAtt?.checkIn]);
```

```
const interval = setInterval(updateTimer, 1000);
return () => clearInterval(interval);
}, [todayAtt?.checkIn, todayAtt?.checkOut]);
```

Sprint Deliverables: 1-Click State Machine

Tasks feature an inline state progression engine directly on dashboard cards:

[Pending] ———(1-Click)————> [In Progress] ———(1-Click)————> [Completed ✓]

Clicking the status badge invokes `handleTaskStatusToggle(taskId, currentStatus)`, triggering `PATCH /api/tasks/:id/status` and optimistically updating the local React state for instantaneous user feedback.

4. Database Design & Quality Assurance

Database Schema & Indexing Strategy

Collection	Key Fields & References	Indexes & Optimization Rationale
Users	firstName, lastName, email, phone, password, role, department, status, isVerified, documents	{ email: 1 } (Unique index for fast authentication lookups) { department: 1 } (Indexed for squad grouping)
Attendance	employee (ref User), date, status, checkIn, checkOut, notes	{ employee: 1, date: 1 } (Unique compound index guaranteeing exactly 1 record per employee per day) { date: 1, status: 1 } (Indexed for daily attendance calculations)
Tasks	title, description, assignedTo (ref User), createdBy (ref User), priority, status, dueDate	{ assignedTo: 1, status: 1 } (Indexed for personal task lists) { dueDate: 1 } (Indexed for deadline sorting)

File Uploads with Multer

- Handled in backend/src/middleware/uploadMiddleware.js using multer.diskStorage.
- MIME Whitelist:** JPEG, PNG, WEBP for photos (Max 2MB); PDF, JPEG, PNG for documents (Max 5MB).
- Stored in backend/uploads/profiles/ and backend/uploads/documents/ with collision-resistant timestamps: [fieldname]-[timestamp]-[random][ext].
- Direct dossier view on /employees/[id] with downloadable file attachments.

Automated Verification: 24/24 Test Suite Pass

✓ **Verified 100% Green (node backend/src/utlis/testApi.js)**

All 24 automated backend test scenarios pass with zero errors:

- Health check returns success
 - Admin & Employee dashboard metrics
 - Tasks & Attendance populated relations
 - Auth /me and profile update
 - Primary Admin (Surender Dubey) login
 - Twilio test number (9582514339) duplicate
- Admin & Employee JWT logins
 - RBAC 403 Forbidden enforcement
 - Forgot password token flow
 - Task status patch transitions
 - Phone number starting with 0 rejected
 - Change verification phone flow

Production Build Verification

Next.js 16 compiled cleanly with Turbopack across all **18 routes** (`npm run build`) with zero JSX or syntax errors.

5. Manager Q&A Defense Guide

Review these answers before presenting to your engineering manager.

Q1: "Walk me through the architecture of EmployeeHub."

"EmployeeHub is built on a modern decoupled architecture. The frontend is built with Next.js 16 and Tailwind CSS using the App Router. The backend is an Express.js REST API with Mongoose and MongoDB Atlas. Incoming requests pass through an authorization pipeline that decodes JWT Bearer tokens and applies strict RBAC route middleware. Controllers fetch and aggregate scoped data from MongoDB, providing clean JSON responses."

Q2: "Why did you choose LocalStorage for JWTs instead of HTTP-Only Cookies?"

"In production, our frontend is hosted on Vercel and our backend API is hosted on Render across two separate domains. Modern web browsers block third-party cookies across different domains under policies like Safari ITP and Chrome cookie restrictions. By storing the token in LocalStorage and transmitting it in the standard Authorization Bearer header, we achieve 100% reliable cross-origin communication, completely eliminate CSRF vulnerabilities, and make the API natively ready for mobile apps."

Q3: "How does the shift punch clock work without overloading the server?"

"We implemented a zero-polling client-side time delta model. When an employee clocks in, the server stores the checkIn timestamp once in MongoDB. On dashboard load, the client receives that timestamp and computes the elapsed delta locally every second using setInterval. This provides a running clock with zero database queries and zero network traffic."

Q4: "How do you prevent unauthorized users from performing administrative actions?"

"We apply defense-in-depth: First, at the API level, every endpoint is protected by our requireRole middleware. Even if an employee intercepts API calls in DevTools and sends requests to admin routes, the backend verifies their role from the signed JWT and responds with HTTP 403 Forbidden. Second, at the UI level, our useAuth hook conditionally displays navigation, action buttons, and sensitive management columns based on user role."

Q5: "How can this project be deployed and scaled?"

"The frontend is optimized for deployment on Vercel with automated global Edge CDN caching and CI/CD. The backend can be deployed on a persistent container platform like Render or Railway. MongoDB Atlas provides replica sets and automatic scaling. For horizontal scaling, local file storage can be migrated to AWS S3, and Redis can be introduced for session caching."